

CAL - Complexidade de Algoritmos

Lista 1

1. Encontre a fórmula fechada dos seguintes somatórios.

- | | |
|--------------------------|--------------------------------|
| a) $\sum_{i=1}^n 2(i-1)$ | e) $\sum_{i=1}^n \frac{1}{i}$ |
| b) $\sum_{i=1}^n 2i - 1$ | f) $\sum_{i=1}^n (a + (i-1)d)$ |
| c) $\sum_{i=1}^n 2i$ | g) $\sum_{i=1}^n ar^i$ |
| d) $\sum_{i=1}^n r^i$ | |

2. Para cada um dos trechos de código a seguir, analise o tempo estimado de execução no melhor e pior caso. Sem notação O . Considere que as variáveis n , m e *vetor* sejam dados de entrada

- | | |
|---|--|
| a) <pre>int a = 0; for(int i = 0; i < n*2; i++){ a = a + n*n; }</pre> | d) <pre>int a[N][N], b=0; for(int i = 0; i < N; i++){ for(int j = 0; j < N; j++){ b = a[i][j]; } }</pre> |
| b) <pre>int s1 = 0; for(int i=0; i < n; i++){ s1 = s1 + 1; }</pre> | e) <pre>int a[N][M], b=0; for(int i = 0; i < N; i++){ for(int j = 0; j < M; j++){ b = a[i][j]; } }</pre> |
| c) <pre>int a = 0; for(int i=0; i < n; i++){ a = a + 1 } for(int j=0; j < n; j++){ a = a + n*j; }</pre> | |

3. Verifique se cada item abaixo é verdadeiro ou falso.

- | | |
|-----------------------------|---------------------------------|
| a) $10^{20}.n^2 \in O(n^2)$ | f) $n \in O(n^3)$ |
| b) $10^{50}.n^2 \in O(n^3)$ | g) $10 \log n \in O(\log(n^2))$ |
| c) $10^{50}.n^2 \in O(n)$ | h) $\log 2n \in \log 3n$ |
| d) $2^{n+1} \in O(2^n)$ | i) $n! = O(2^n)$ |
| e) $2^{2n} \in O(2^n)$ | |

4. Faça uma análise de complexidade para o pior e melhor caso dos algoritmos abaixo.

```
a) int soma(int n){
    int soma = 0;
    for(i=1; i<n+1; i++){
        soma += i;
    }
    return soma;
}
```

```
b) for(i=0; i<n-1; i++){
    for(j=i+1; j<n; j++){
        if(A[i] == A[j]){
            return 0;
        }
    }
}
return 1;
```

```
c) def prodMat(m1, m2):
    n = len(m1)
    res = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                res[i][j] += m1[i][k] * m2[k][j]
    return res
```

```
d) def encontrar_pares_soma(arr, soma):
    pares = []
    n = len(arr)
    for i in range(n):
        for j in range(i + 1, n):
            if arr[i] + arr[j] == soma:
                pares.append((arr[i], arr[j]))
    return pares
```

```
e) int customAlg(int *v, int n){
    int i, soma = 0;
    for(i=0; i<n; i=i+2){
        soma = soma + vetor[i];
    }
    return soma;
}
```

```
f) int customAlg2(int *v, int n){
    int i, soma = 0;
    for(i=0; i<n; i=i+2){
        soma = soma + vetor[i];
    }
    for(i=1; i<n; i=i+2){
        soma = soma + vetor[i];
    }
    return soma;
}
```

5. Podemos analisar a complexidade de um algoritmo apenas contando o número de execuções das suas instruções. Mostre a complexidade dos algoritmos a seguir utilizando este método e mostre como obteve sua resposta:

- a) Algoritmo de Ordenação Bubble-Sort.
- b) Algoritmo de pesquisa num vetor.

6. Determine a complexidade do código a seguir:

```
def fatorial(n):
    if n == 0:
        return 1
    return n * fatorial(n - 1)
```

7. Implemente um algoritmo de Fibonacci não recursivo e determine a sua complexidade.

8. Mostre que:

- a) A solução de $T(n) = T(\frac{n}{2}) + 1; T(1) = 1$ é $O(\log n)$
- b) A solução de $T(n) = 2T(\frac{n}{2}) + n; T(1) = 1$ é $O(n \log n)$

9. Considere o seguinte algoritmo:

```
def astec(n, c1=' ', c2=' '):
    if n > 0:
        print(c1, end=" ")
        astec(n-1)
        astec(n-1)
        print(c2, end=" ")
    else:
        print("*", end=" ")
```

Determine quantas vezes a função astec é chamada em função de n .

10. Analise o algoritmo a seguir:

```
def algoritmo_desconhecido(lista):  
    n = len(lista)  
    for i in range(n):  
        for j in range(1, n, 2):  
            if lista[i] < lista[j]:  
                temp = lista[i]  
                lista[i] = lista[j]  
                lista[j] = temp
```

- a) Qual o propósito desse algoritmo?
- b) Faça a análise da complexidade do mesmo.

Respostas

1. ...

2. a) $1 + 2n$

d) $1 + N^2$

b) $1 + n$

c) $1 + 2n$

e) $1 + 1 + NM$

3. a) $10^{20} = c.n^2$.

e) Falso

Se $c = 10^{20}$

então ficará igual. Verdadeira

f) Verdadeira

b) Verdadeira

g) Verdadeira

c) Falso

h) Verdadeira

d) Verdadeiro

i) Falso

4. a) Melhor: $\Omega(1)$; Pior: $O(n)$

d) Melhor: $\Omega(1)$; Pior: $O(n^2)$

b) Melhor: $\Omega(1)$; Pior: $O(n^2)$

e) Melhor: $\Omega(1)$; Pior: $1 + \frac{n}{2} \Rightarrow O(n)$

c) Melhor: $\Omega(1)$; Pior: $O(n^3)$

f) Melhor: $\Omega(1)$; Pior: $1 + 2\frac{n}{2} \Rightarrow O(n)$

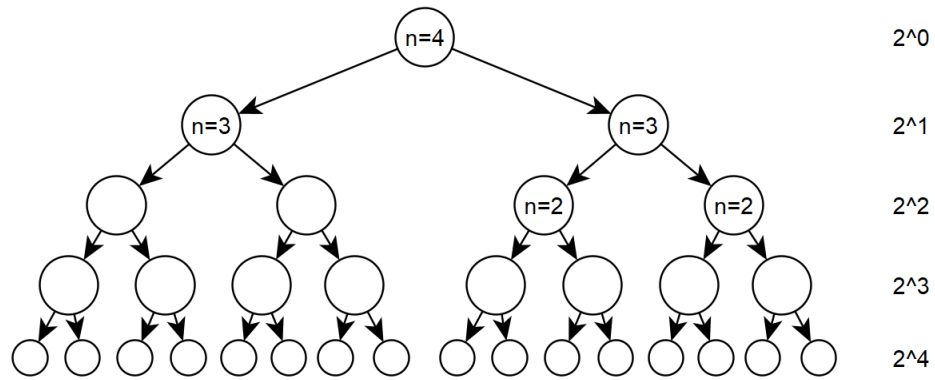
5. -

6. $O(n)$

7. -

8. -

9. Podemos analisar as chamadas recursivas através de uma árvore e deduzir quantas chamadas são feitas ao total. Por exemplo, se $n = 4$, então teremos o somatório $2^0 + \dots + 2^4 = 32$. O que corresponde a **soma de 5 termos de uma PG**.



Portanto, a resposta é a fórmula do somatório de uma PG. Nesse caso, a razão é 2, o primeiro termo é 1 e o último é $n+1$ (pois com $n=1$ ainda há duas chamadas).

Mostre essa fórmula na sua resposta.

10. a) -
b) $O(n^2)$